

StockMate — Reflection

A simple stock tracker for a small motel • C#, Windows Forms, SQLite

Aashir Bhusal | s2400901
ITS203 Assessment C — Milestone 3 (15%)
Final submission, October 2026

1. Project summary

StockMate is a Windows desktop program for a small motel that still tracks its storeroom on paper. Paper counts cause two problems: running out of things like towels in the middle of a shift, and over-ordering stock that then goes out of date. StockMate keeps one count for every item. Staff record a **receipt** when a delivery arrives and an **issue** when stock leaves the storeroom, and each one is saved with the date and their name. When the program opens it lists what needs attention: items at or below their reorder level, consumables near or past their use-by date, and durables that are due for replacement. Any item's history can be viewed, the list can be exported to a CSV file with its total value, and an item that is no longer stocked can be deactivated without losing its history. The data is kept in a SQLite file with two tables. All eight functional requirements in my proposal are now built.

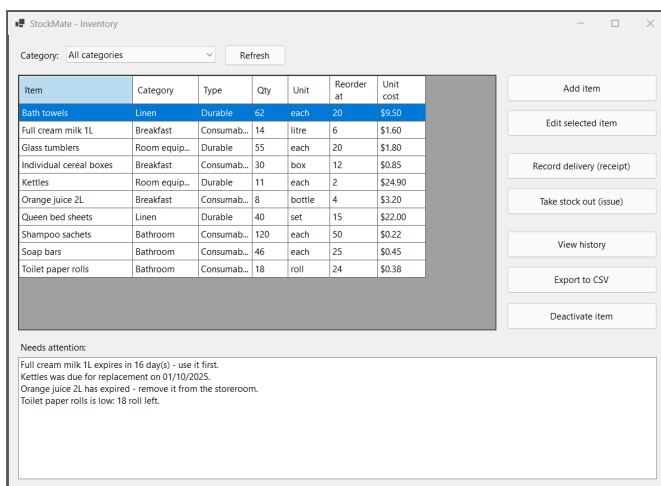


Figure 1 — Main window: stock list, filter, actions and start-up warnings

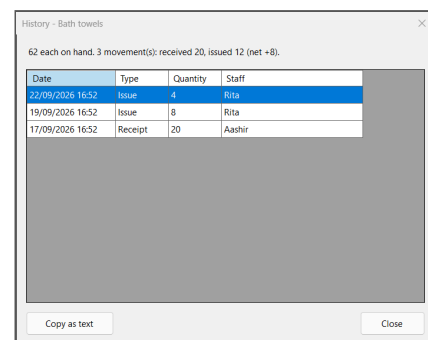


Figure 2 — History of one item

2. GitHub repository link

<https://github.com/AashirBhusal/StockMate>

The repository is public. It holds the Visual Studio solution, the README, my proposal, my progress report, this reflection, my development notes and the test record (docs/TESTING.md).

3. OOP design explanation

Principle	How my code shows it
Classes and objects	Six classes do the work: <code>StockItem</code> , <code>ConsumableItem</code> , <code>DurableItem</code> , <code>StockMovement</code> , <code>Database</code> and <code>CsvExporter</code> . Every row of the <code>StockItems</code> table becomes one object: <code>Database.ReadItem()</code> reads the <code>ItemType</code> column and builds a <code>ConsumableItem</code> or a <code>DurableItem</code> . Every row of <code>Transactions</code> becomes a <code>StockMovement</code> .
Encapsulation	<code>StockItem.QuantityOnHand</code> has a private setter. Nothing outside the class can assign to it; it only changes through <code>Receive()</code> and <code>Issue()</code> , which check the rules first. <code>Name</code> , <code>Category</code> , <code>ReorderLevel</code> and <code>UnitCost</code> keep their values in private fields and validate in the setter. In <code>StockMovement</code> the type, quantity, date and staff name are get-only properties set once in the constructor, because a movement is a record of something that has already happened.
Inheritance	<code>ConsumableItem</code> and <code>DurableItem</code> both extend <code>StockItem</code> . They inherit the shared fields, the validation, <code>Receive()</code> , <code>Issue()</code> , <code>IsLowStock</code> and <code>TotalValue</code> , and add only what is different: a use-by date for one, a replacement interval and last-replaced date for the other. Each constructor passes the shared values up with: <code>base(name, category, quantity)</code> .
Polymorphism	Both subclasses override <code>NeedsAttention()</code> , <code>AttentionMessage()</code> and <code>ToReportLine()</code> . <code>MainForm.ShowWarnings()</code> loops over a <code>List<StockItem></code> and calls the first two without checking the type, and <code>CsvExporter.Export()</code> reuses the same calls for its <code>Status</code> column. Through the interface, <code>HistoryForm.btnCopy_Click()</code> puts an item and its movements into one <code>List<IReportable></code> and calls <code>ToReportLine()</code> on each.
Abstraction	<code>StockItem</code> is abstract, because a plain stock item cannot answer whether it needs attention today; it declares <code>ItemType</code> , <code>NeedsAttention()</code> and <code>AttentionMessage()</code> as abstract and leaves the answer to the subclasses. <code>IReportable</code> is an interface and not a base-class method because <code>StockMovement</code> is not a stock item but still needs a report line. The forms never see SQL: they call methods such as <code>GetHistory()</code> on <code>Database</code> .

Principle	How my code shows it
Exception handling	Issue() throws my own <code>InsufficientStockException</code> , which carries the item name, the quantity requested and the quantity available. <code>MainForm.RecordMovement()</code> catches it in its own block, placed before <code>catch (Exception)</code> , so a shortage gets a helpful message and real errors are still reported. <code>ItemForm</code> catches the <code>ArgumentException</code> thrown by the setters. The export catches <code>IOException</code> for a file that is open in Excel. <code>Program.Main()</code> wraps start-up, so a damaged database shows a message, not an error screen.

Table 1 — The six OOP principles and where they are in my code

4. Reflection on development

What changed from my proposal. Four things. The database went from three tables to two: I dropped the separate `Categories` table because it meant a join on every read and an extra form, for no benefit at this size. The proposal put everything in one window with tabs; I built one main window with small pop-up forms, which keeps the stock list in view. FR-07 and FR-08 were listed as extras and both are now built, and the history window needed a class that was not in my proposal at all, `StockMovement`. Last, one sentence of the proposal turned out not to be true of my own program: it said the database would be saved next to the program, and until the final week it was not (section 5).

What I learned. The clearest lesson was about interfaces. I wrote `IReportable` in August, expecting transactions to need it later, and then for six weeks nothing in the program called it. It only became useful when the history window had to treat an item and its movements the same way, and at that point it was exactly the right tool, because the two classes share no base class. An abstraction is only worth having once something uses it. I also saw encapsulation pay off during testing: every refused issue left the quantity and the database untouched, and it could not have been otherwise, because `Issue()` is the only way to lower a quantity. Polymorphism saved real work too. The CSV export needed a status for each row, and it got one from the two calls the main window already used, without one line of code about item types.

What was most challenging. The hardest problems were not in the class design but at the edges, where my code meets Windows. Setting up .NET without administrator rights was the first of these. The two failures that testing found were both of this kind: a file name that Windows resolves against the folder a program was started from, and a number box that throws when it is given a value above its maximum. None of these show up in a class diagram, and I only found them by running the program in ways I had not planned for.

What I would do differently. First, I would test from the first week, not the last. Both failures had been in the code since August and would have been cheaper to fix then. Second, I would work more steadily. My commit history shows four blocks of work (23 August, 9 September, 21 to 23 September and 4 October) and not a little each week. In my progress report I said each remaining feature would get its own commit on the day I finished it. Each feature did get its own commit, but the last block was again done in one sitting and later than I had planned. Third, I would not reload the whole list after every change. That is the reason the program misses its speed target at 2,000 items, and updating only the row that changed should fix it. Fourth, I would write unit tests for the `Models` classes, so the rules could be checked without opening a window.

5. Testing summary

The program was tested on 4 October with 38 scenarios, each carried out through the real windows against a separate test database so the demo data was not touched. A script did the clicking and typing so that the steps could be repeated exactly, and after each group the database was read with a separate query to confirm that what the screen showed had really been saved. The full record is in `docs/TESTING.md`.

Area	Tests	Examples	Result
Adding and editing items	10	Blank name, name of only spaces, blank category; type and quantity locked when editing; deactivate answering No then Yes	All passed
Start-up warnings	11	Use-by in 30 days warns and 31 does not; quantity equal to the reorder level warns and one above does not	All passed
Receipts and issues	6	Issue 100 with 82 on hand is refused and nothing is saved; issue exactly 82 ends at zero; no staff name	All passed
History, filter and export	7	Export total of \$3,985.50 matched a separate SQL calculation; a name containing a comma was quoted; a locked file gave a message	All passed
Keeping the data safe	3	Close and reopen; start from a different folder; damaged database file	1 failed, fixed, passed
Very large quantity	1	Receive 100,000, then press Edit	Failed, fixed, passed

Table 2 — Test scenarios by area

What testing found. Two real failures. Started from a different folder, the program showed an empty list and quietly created a second database there; it now builds the path from the program's own folder. Pressing Edit on an item holding more than 100,000 crashed the program, because the read-only quantity box still had a maximum of 100,000; the form now raises the maximum first. The wording "expires in 0 day(s)" now says "expires today". Earlier, preparing my progress report had already shown up a label covering the quantity box on the movement form. Against my non-functional requirements, a damaged database gives a plain message and start-up with 2,000 items took about 0.8 seconds in the first set of runs, though a later repeat on a busy machine was slower. Refresh, and so every saved movement, took 1.4 seconds or more at that size. That misses my one-second target and I have not fixed it. Two limits of the testing itself: the date pickers were not driven by the script, so the date cases were set up directly in the test database, and everything was run on one computer.

6. References and tools used

Microsoft. (n.d.). *Desktop Guide (Windows Forms .NET)*. <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>

Microsoft. (n.d.). *Microsoft.Data.Sqlite overview*. <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/>

SQLite. (n.d.). *Appropriate uses for SQLite*. <https://www.sqlite.org/whentouse.html>

Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6* (11th ed.). Apress.

Tools. .NET 8 SDK, Windows Forms, Microsoft.Data.Sqlite 8.0.8, Visual Studio Code, Git and GitHub.

Generative AI. I used Claude (Anthropic) as study support on this project: to talk through the class design, to help write and comment the `Models` classes, and to check my code for mistakes. I have read and understood all of it and can explain every part. The same disclosure appears in my README.